VELTRI TECNOLOGIC

Automatización inteligente de procesos

Semana 10

Informe académico de las cinco actividades del ecosistema LangChain

Ingeniería de Sistemas

31 de octubre de 2026 · Lima, Perú

Estado de la entrega

Las demostraciones locales de LCEL, HITL nativo y Deep Agents cuentan con resultados verificables. La suite registrada aprobó 464 pruebas. La ejecución remota de LangSmith permanece pendiente de acreditación visual; este informe no la presenta como completada.

Las capturas auténticas de pantalla siguen pendientes de incorporación. Se incluyen diagramas derivados del código, extractos exactos y registros originales, identificados según su naturaleza.

El flujo comercial principal y las demostraciones académicas se describen por separado. Los precios, existencias y casos utilizados son ejemplos sintéticos.

2 Introducción

Veltri Tecnologic aborda la atención de consultas de hardware mediante agentes por dominio, herramientas deterministas y un orquestador LangGraph. La propuesta combina interpretación estructurada, validación técnica, consulta de inventario y preparación de borradores, con revisión humana cuando la evidencia impide continuar.

Este informe desarrolla exactamente las cinco actividades indicadas por el profesor: identificación de LCEL; grafo e HITL frente al BPMN TO-BE; monitoreo LangSmith; Deep Agents; y cuatro componentes del ecosistema. El análisis se apoya en el código y las ejecuciones conservadas el 31 de octubre de 2026.

3 Objetivos

- Identificar componentes LCEL con archivo, línea y aplicación comprobable.
- Explicar el grafo real, demostrar HITL y comparar su comportamiento con el proceso BPMN.
- Delimitar el estado de la traza LangSmith sin confundir aceptación HTTP con verificación remota.
- Demostrar Deep Agents reales con planificación y delegación bajo MockLLM.
- Relacionar Deep Agents, LangChain y LangGraph con LangSmith como observabilidad transversal.

Criterio de evidencia

Se distinguen cuatro categorías: código implementado, ejecución local registrada, diagrama derivado y captura de pantalla. Una categoría no sustituye a otra. Los registros HTTP 202 no prueban por sí solos que el árbol remoto esté completo.

4 Descripción de Veltri

La aplicación comercial utiliza FastAPI, una interfaz web local, agentes personalizados A1–A8, herramientas en src/tools y recuperación documental con tenant explícito. Las cantidades, precios, compatibilidad y stock se calculan mediante código, no mediante el modelo de lenguaje.

Tabla 1. Agentes del flujo comercial

Agentes	Dominio y responsabilidad	Alcance real
A1–A3	D1: interpretar, armar y cotizar	Interpretación estructurada y herramientas locales
A4	D2: compatibilidad	Validación técnica determinista
A5	D3: inventario	Lectura de stock sintético
A6–A8	D4: pagos, borradores y seguimiento	Comparación local; sin proveedor de pagos

Separación de las demostraciones

src/academic contiene LCEL, HITL nativo y Deep Agents académicos. Estos módulos no están expuestos por la API ni reemplazan los agentes comerciales. El script demo_academica.py desactiva las trazas de su proceso y registra resultados sintéticos locales.

Tabla 2. Límites entre aplicación y demostración

Componente	Flujo comercial	Demo académica
Agentes	Clases BaseAgent A1–A8	create_deep_agent y dos especialistas
HITL	Bandeja local sin pausa nativa	interrupt + InMemorySaver + Command
Modelo	Fábrica get_llm	MockLLM programado con adaptador
Efectos	Comparaciones y borradores locales	Ninguna autorización comercial

El inventario conserva 84 SKUs de ejemplo en soles. CPU-AMD-5500 tiene stock cero y se usa para evidenciar un bloqueo de inventario. Ningún dato se presenta como precio o disponibilidad real de una tienda.

5 Actividad 1 LCEL

LCEL permite componer objetos Runnable. En Veltri se implementó una cadena determinista que valida una entrada cerrada, ejecuta ramas paralelas y proyecta una salida tipada. La composición con | construye un RunnableSequence real, comprobado en tests/test_academico.py.

Código · src/academic/lcel.py, líneas 3–20

```
3  from langchain_core.runnables import RunnableLambda, RunnableParallel
4
5  from src.schemas.academico import EntradaAcademica, SalidaAcademica
6  from src.tools.catalogo import Catalogo
7  from src.tools.inventario import comprobar_stock
8
9
10 def crear_pipeline():
11     """Validación -> ramas paralelas -> contrato de salida mediante operador LCEL."""
12     return (
13         RunnableLambda(EntradaAcademica.model_validate)
14         | RunnableParallel(
15             escenario=RunnableLambda(lambda entrada: entrada.escenario),
16             revision=RunnableLambda(lambda _: comprobar_stock(
17                 Catalogo(), {"CPU-AMD-5500": 1})["hitl_requerido"]),
18         )
19         | RunnableLambda(lambda ramas: SalidaAcademica(requiere_revision=ramas["revision"]))
20     )
```

Tabla 3. Identificación de términos LCEL

Término	Aplicación y ubicación
RunnableLambda	Envuelve validación, selección de escenario, stock y salida; lcel.py:13–19.
RunnableParallel	Produce un diccionario con escenario y revisión; lcel.py:14–18.
RunnableSequence y |	Encadena las etapas; lcel.py:12–20; test_academico.py:15–18.
invoke	Ejecuta la cadena con una entrada; scripts/demo_academica.py.
Pydantic	Valida contratos; no se identifica como parser LCEL independiente.
ChatPromptTemplate	No utilizado: la operación determinista no necesita un prompt.

Actividad 1 Ejecución y evidencia

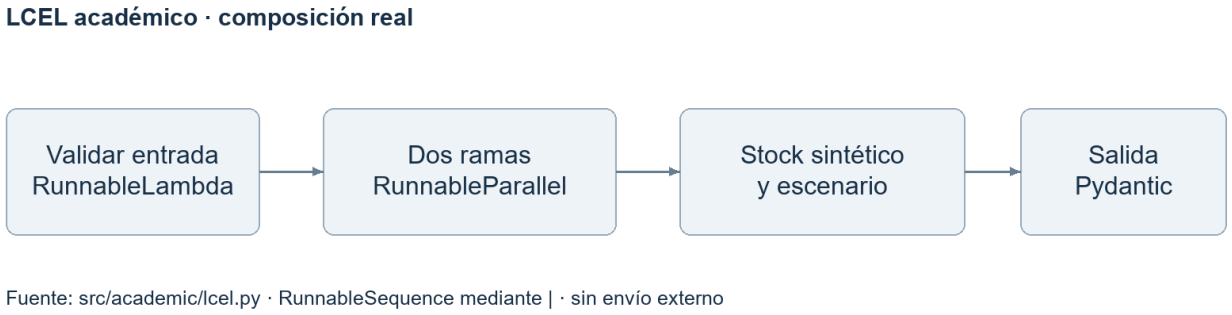


Figura 1. Pipeline académico de revisión de stock. Fuente: src/academic/lcel.py. Diagrama elaborado desde evidencia; no es captura de pantalla.

La rama de revisión llama a comprobar_stock en src/tools/inventario.py sobre el catálogo local. El cálculo queda en la herramienta determinista. La otra rama conserva el escenario. La etapa final construye SalidaAcademica; no redacta una cotización ni modifica el inventario.

Entrada y salida registradas

Entrada: {}. Se aplica el escenario predeterminado revision_sintetica, sin mensajes libres.

```
{
  "sintetico": true,
  "requiere_revision": true,
  "efecto_comercial": false
}
```

Fuente: ejecuciones.json, campo lcel. La salida confirma revisión requerida y ausencia de efecto comercial. Las pruebas rechazan campos adicionales y escenarios distintos; también comprueban el tipo RunnableSequence y el contrato resultante.

Límite de la evidencia

El pipeline tiene dos ramas paralelas reales, aunque la proyección final utiliza solo el resultado de revisión. No demuestra una composición prompt | model | parser ni una llamada de inferencia. Falta la captura auténtica del código y de su salida en un editor o terminal del usuario.

6 Actividad 2 LangGraph

El grafo compilado comercial contiene 12 nodos operativos, 14 vértices al contar START y END, y 20 aristas. La seguridad y A1 se ejecutan antes del grafo. validar_plan selecciona una sola ruta tipada; las ramas convergen en decidir, responder y END.

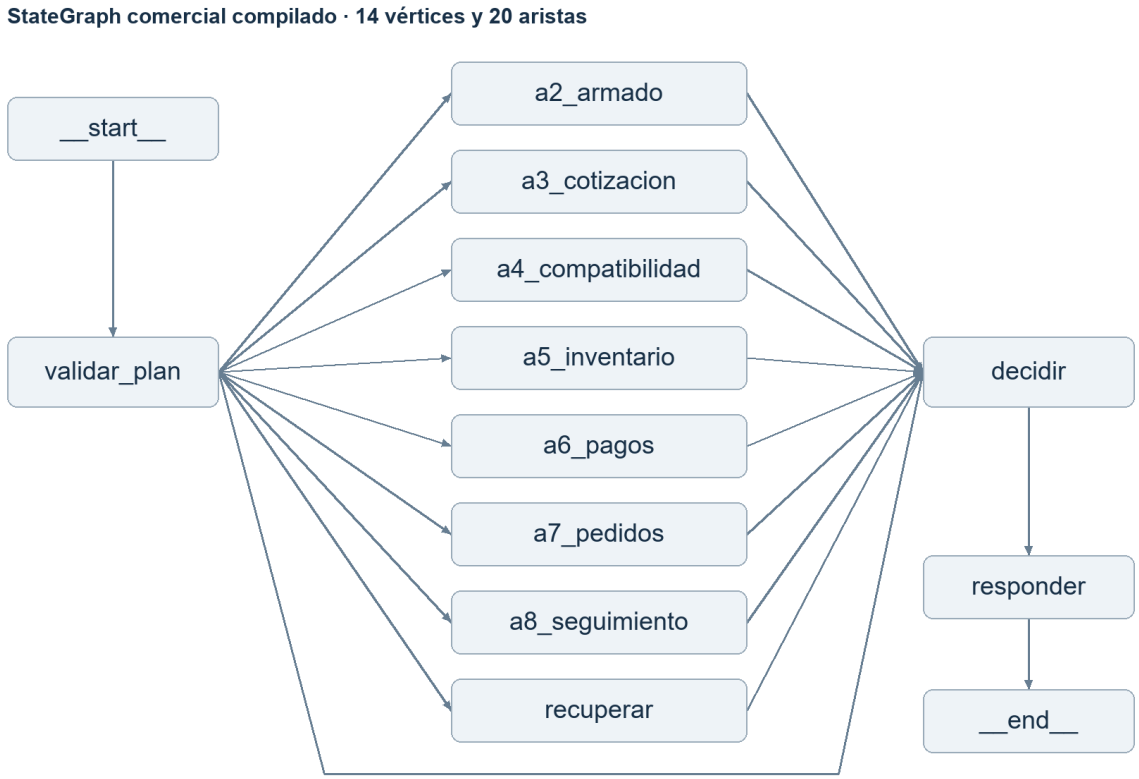


Figura 2. Nodos y aristas del grafo comercial. Fuente: grafo-comercial.json; src/graph/workflow.py:28–53. Diagrama elaborado desde evidencia; no es captura de pantalla.

La rama recuperar representa la consulta documental con tenant. El selector también puede pasar directamente a decidir. Las aristas condicionales provienen del grafo compilado; la presencia de una ruta no implica que cada consulta la recorra.

El diagrama se generó a partir de los nodos y aristas almacenados por get_graph(). No se modificó el StateGraph para acercarlo artificialmente al BPMN.

Actividad 2 HITL nativo y bandeja

Código · src/academic/hitl.py, líneas 25–33

```
25      def revisar(estado: Estado):
26          decision = Decision.model_validate(interrupt({"tipo": "revision_sintetica"}))
27          return {"resultado": decision.accion}
28
29      grafo = StateGraph(Estado)
30      grafo.add_node("revision_humana", revisar)
31      grafo.add_edge(START, "revision_humana")
32      grafo.add_edge("revision_humana", END)
33      return grafo.compile(checkpointer=InMemorySaver())
```

HITL nativo académico · pausa y reanudación

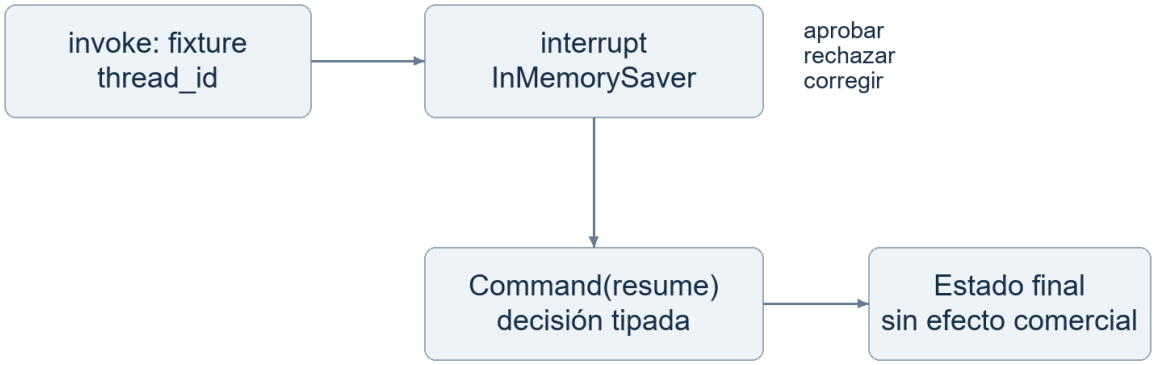


Figura 3. Interrupción y reanudación de la demo aislada. Fuente: src/academic/hitl.py y ejecuciones.json. Diagrama elaborado desde evidencia; no es captura de pantalla.

Tabla 4. Tres caminos ejecutados de HITL nativo

Decisión simulada	Checkpoint pendiente	Resultado
aprobar	revision_humana	aprobar
rechazar	revision_humana	rechazar
corregir	revision_humana	corregir

Command(resume=...) reanuda el mismo thread_id. Las pruebas comprueban la interrupción, el nodo pendiente y la finalización, y rechazan una acción inválida. InMemorySaver conserva el estado solo dentro del proceso; no hay persistencia duradera ni revisor autenticado.

La bandeja comercial permanece separada. La ejecución guardada cambió de pendiente a correccion_solicitada. Resolver esa bandeja no reanuda el StateGraph comercial ni ejecuta una venta.

Actividad 2 Comparación con BPMN TO BE

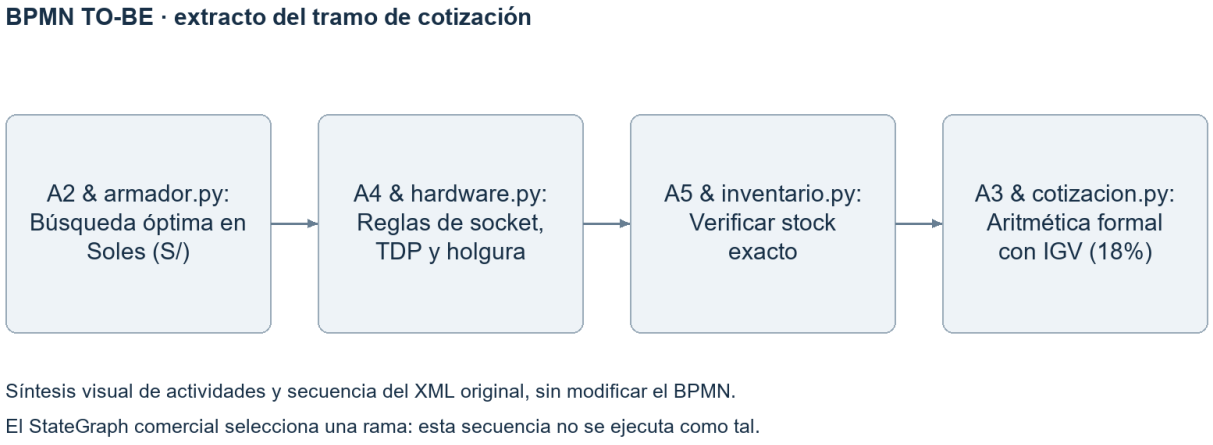


Figura 4. Secuencia de cotización representada por el BPMN original. Fuente: docs/bpmn_to_be.bpmn. Diagrama elaborado desde evidencia; no es captura de pantalla.

Tabla 5. Compatibilidades y discrepancias verificadas

Aspecto	BPMN TO-BE	Implementación y compatibilidad
Cotización	A2 → A4 → A5 → A3	Parcial: el grafo selecciona una rama por solicitud.
Enrutamiento	Gateway inclusivo	Diferencia: selector tipado de una rama.
Seguridad/A1	Actividades del proceso	Previas al StateGraph en el código.
Revisión	Intervención humana	Bandeja local; HITL nativo solo académico.
Pagos/pedidos	Conciliación y registro	Comparación y borradores; sin proveedor externo.
Seguimiento	A8 no representado	A8 sí existe en el sistema.
IGV y tiempos	IGV 18 % y objetivo <1,5 s	No usar el BPMN como prueba del cálculo ni del rendimiento.

La compatibilidad es parcial. Las políticas de garantía, el cálculo de IGV y las diferencias de tiempos siguen fuera del alcance de esta fase. El XML original se conserva íntegro; la figura es un extracto explicativo, no una captura de un modelador BPMN.

7 Actividad 3 Monitoreo con LangSmith

Estado: evidencia remota pendiente de acreditación. El segundo intento autorizado registró nueve creaciones y nueve cierres aceptados por HTTP 202. La lectura posterior recibió HTTP 403 y se detuvo sin reintentos. No se verificó un run_id ni el árbol remoto.

Tabla 6. Resultado real del intento remoto

Etapa	Resultado observado	Qué acredita
POST /runs	9 respuestas 202	Aceptación de solicitudes de creación
PATCH /runs/{run_id}	9 respuestas 202	Aceptación de cierres
GET /runs/{run_id}	403; categoría permisos	Verificación remota bloqueada
run_id / duración / árbol	No verificados	No disponibles para acreditar la actividad

Fuente: docs/evidencias/semana10-fase43/ejecucion-unica.json. Ventana del procedimiento: 09:39:46–09:40:03, America/Lima, 31/10/2026. Ese intervalo no es la duración remota de la traza. El identificador de correlación HTTP no es el run_id.

Instrumentación y privacidad

Código · src/core/observabilidad.py, líneas 43–55

```
43      return Client(
44          api_url=config.endpoint,
45          api_key=config.clave.get_secret_value(),
46          auto_batch_tracing=False,
47          hide_inputs=True,
48          hide_outputs=True,
49          hide_metadata=lambda _: {"synthetic": True, "model_mode": "mock", "scenario": "stock_demo"},
50          omit_traced_runtime_info=True,
51          timeout_ms=2000,
52          tracing_mode="langsmith",
53          otel_enabled=False,
54          tracing_sampling_rate=1.0,
55          session=TransporteDiagnostico(),
```

La sesión exige doble opt-in y modo mock. Inputs y outputs se mantienen vacíos y los metadatos se restringen a constantes sintéticas. Los dos controles quedaron apagados. No se realizaron nuevos envíos para elaborar este informe.

Evidencia que debe incorporarse

Desde el panel autenticado: proyecto veltri-semana10-demo, ejecución seleccionada, run_id, duración, árbol de nueve spans, metadata y entradas/salidas vacías. Las capturas no están disponibles. Una clave Tracing Only puede explicar falta de lectura, pero la causa del 403 no está confirmada.

8 Actividad 4 Deep Agents generados

El módulo académico importa y utiliza create_deep_agent del paquete deepagents 0.4.11. El coordinador y sus dos especialistas son distintos de A1–A8. El adaptador BaseChatModel obtiene MockLLM desde get_llm y traduce fixtures tipados a llamadas de herramientas.

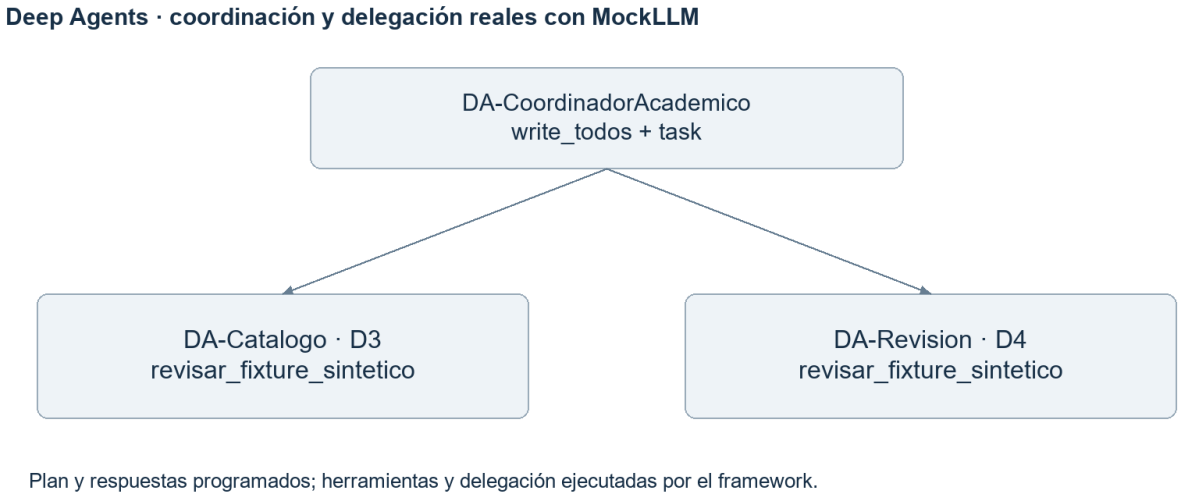


Figura 5. Coordinador y especialistas del framework. Fuente: src/academic/deep_agents.py. Diagrama elaborado desde evidencia; no es captura de pantalla.

Tabla 7. Deep Agents implementados

Nombre	Responsabilidad	Herramientas
DA-CoordinadorAcademico	Organizar la revisión sintética	write_todos y task
DA-Catalogo	D3: revisar evidencia sintética	revisar_fixture_sintetico
DA-Revision	D4: explicar revisión requerida	revisar_fixture_sintetico

Los nombres y responsabilidades fueron aprobados por el usuario. No aprueban ventas ni pagos. La herramienta lee el fixture de inventario y devuelve únicamente atributos sintéticos reducidos; no reserva ni modifica stock.

Código · src/academic/deep_agents.py, líneas 86–89

```
86      return create_deep_agent(
87          model=AdaptadorMock(pasos, auditoria=auditoria), tools=[], subagents=subagentes,
88          system_prompt="DA-CoordinadorAcademico: coordinación académica sin efectos comerciales.",
89      )
```

Actividad 4 Planificación y delegación verificadas

Código · src/academic/deep_agents.py, líneas 62–83

```
62      herramienta = tool(revisar_fixture_sintetico)
63
64      pasos = [{"herramienta": "write_todos", "argumentos": {"todos": [
65          {"content": "Revisar evidencia sintética", "status": "in_progress"},
66          {"content": "Explicar bloqueo", "status": "pending"}]}}]
67      subagentes = []
68      for nombre, descripcion in [
69          ("DA-Catalogo", "Revisar evidencia sintética, dominio D3"),
70          ("DA-Revision", "Explicar bloqueo de revisión, dominio D4"),
71      ]:
72          pasos.append({"herramienta": "task", "argumentos": {
73              "subagent_type": nombre, "description": descripcion}})
74          subagentes.append({"name": nombre, "description": descripcion,
75                             "system_prompt": "Solo evidencia sintética; sin efectos comerciales.",
76                             "model": AdaptadorMock([
77                                 {"herramienta": "revisar_fixture_sintetico", "argumentos": {}},
78                                 {"contenido":
79                                 '{"sintetico":true,"requiere_revision":true}'}],
80                                 auditoria=auditoria, rol=nombre),
81                             "tools": [herramienta]})
82      pasos += [{"herramienta": "write_todos", "argumentos": {"todos": [
83          {"content": "Revisar evidencia sintética", "status": "completed"},
```

Secuencia registrada

El coordinador emitió write_todos con dos tareas, delegó mediante task en DA-Catalogo y DA-Revision, marcó ambas tareas como completed y devolvió la salida académica. Cada especialista ejecutó la herramienta y recibió una respuesta con estado success.

Tabla 8. Plan, delegación y herramientas en ejecuciones.json

Evidencia	Resultado real
Llamadas task	DA-Catalogo y DA-Revision
Plan final	Dos tareas completed
Auditoría de especialistas	revisar_fixture_sintetico: success en ambos
Salida final	sintetico=true; requiere_revision=true; efecto_comercial=false

La delegación la ejecuta el framework; el modelo y el plan son programados. Se verifica integración funcional sin pagar inferencia. No se demuestra planificación autónoma, memoria persistente ni utilización del filesystem virtual. Los mensajes y todos son contexto de esta ejecución.

Las pruebas verifican las delegaciones, los cuatro ToolMessage del coordinador, la ausencia de errores, el plan completado y la auditoría exitosa de cada especialista. La captura de pantalla del código y de la ejecución permanece pendiente.

9 Actividad 5 Cuatro componentes del ecosistema

Ecosistema LangChain aplicado a Veltri

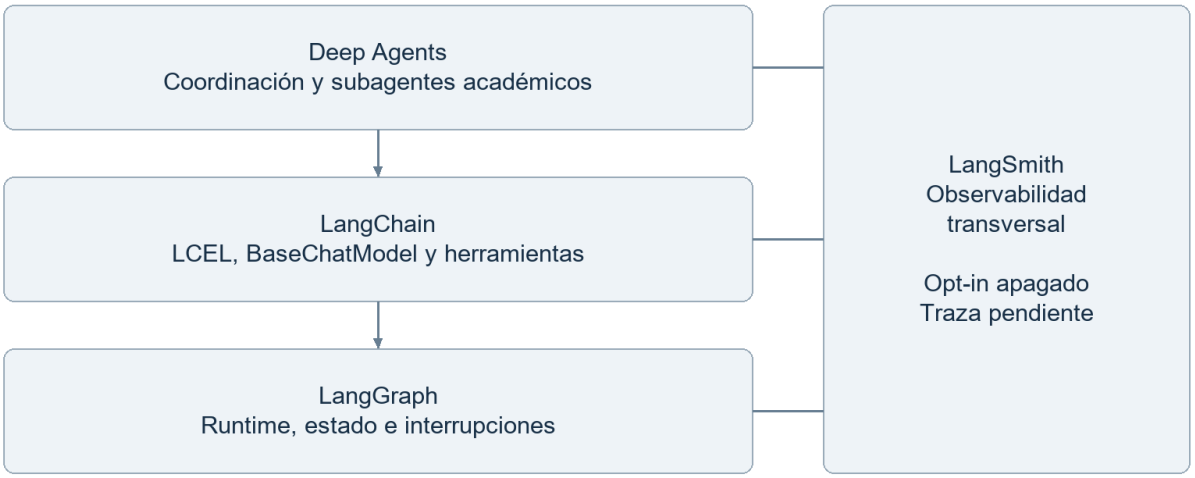


Figura 6. Tres capas de agentes y observabilidad transversal. Fuente: src/academic y src/core/observabilidad.py. Diagrama elaborado desde evidencia; no es captura de pantalla.

Tabla 9. Relación del ecosistema con Veltri

Componente	Función y aplicación real
Deep Agents	Coordinación de alto nivel con planificación y especialistas; deep_agents.py.
LangChain	Primitivas de modelos y herramientas; BaseChatModel y cadena LCEL; lcel.py.
LangGraph	Runtime del grafo comercial y del agente; interrupciones en hitl.py.
LangSmith	Observabilidad transversal mediante instrumentación opt-in; verificación remota pendiente.

Deep Agents se apoya en LangChain y LangGraph. LCEL organiza una operación determinista; LangGraph coordina estado y ejecución. LangSmith puede observar los componentes, pero no es una cuarta capa de ejecución en serie. Build, Test, Deploy y Monitor no sustituyen esta clasificación.

Código · src/academic/deep_agents.py, líneas 6–9

```
6  from langchain_core.messages import AIMessage, ToolMessage
7  from langchain_core.outputs import ChatGeneration, ChatResult
8  from langchain_core.tools import tool
9  from pydantic import PrivateAttr
```

10 Resultados y pruebas

Se utiliza la verificación conservada, sin repetir fases ni ejecutar otra demostración remota. El registro corresponde a python scripts/verificar.py bajo aislamiento de red externa y sin credenciales reales en las pruebas.

Tabla 10. Resultados de verificar.log y suite-summary.json

Comprobación	Resultado registrado
Pytest	464 aprobadas; 0 fallos, 0 errores, 0 omitidas
Ruff	All checks passed!
Secretos	Sin hallazgos
Tiempo pytest	48,51 segundos
Advertencias	2 deprecaciones: ChromaDB y Google SDK
Regresión	456 pruebas previas más 8 casos académicos
Suite completa	Código de salida 0; 58,552 segundos con verificaciones

Salida textual original

```
[ruff] All checks passed!

464 passed, 2 warnings in 48.51s

[secretos] Escaneo de secretos: sin hallazgos.

ruff OK | pytest OK | secretos OK
```

Los ocho casos nuevos comprenden una composición LCEL, dos entradas inválidas parametrizadas, tres decisiones HITL, una decisión no admitida y una ejecución de Deep Agents. La línea base protegida no se actualizó.

Versiones ejecutadas

deepagents 0.4.11; langchain 1.2.11; langchain-core 1.6.12; langgraph 1.2.16; langsmith 0.14.11.

Los resultados son evidencia de las pruebas ejecutadas en esa fecha, no una garantía de exactitud comercial completa. La discrepancia BPMN y la acreditación LangSmith pendiente siguen explícitas.

11 Conclusiones

La implementación demuestra una cadena LCEL auténtica y una operación determinista de revisión de inventario. El contrato tipado impide introducir mensajes arbitrarios en la demo y mantiene el cálculo en herramientas de código.

LangGraph dispone de un grafo comercial identificable y una demostración académica de pausa y reanudación. La bandeja comercial y el mecanismo nativo son distintos. El BPMN TO-BE conserva diferencias relevantes de secuencia, compuertas y alcance operativo.

Los Deep Agents se ejecutan mediante el framework real con planificación y delegación programadas por MockLLM. Esta evidencia valida la integración y las herramientas, sin acreditar autonomía del modelo ni memoria persistente.

Las cuatro partes del ecosistema tienen una correspondencia concreta con el proyecto. LangSmith permanece como observabilidad transversal con instrumentación segura; la actividad de monitoreo no puede declararse acreditada hasta verificar la traza remota existente.

Tabla 11. Matriz final sin exceder la evidencia

Requisito	Conclusión de cumplimiento
1 LCEL	Implementado y ejecutado; capturas pendientes
2 LangGraph/HITL/BPMN	Demostrado localmente; compatibilidad parcial; capturas pendientes
3 LangSmith	Pendiente de acreditación remota
4 Deep Agents	Framework ejecutado con MockLLM; capturas pendientes
5 Cuatro componentes	Documentados y relacionados con archivos reales

La incorporación de capturas auténticas y la comprobación de la traza son las acciones restantes para una entrega íntegramente acreditada. No se requiere otra demostración externa para elaborar este documento.

12 Referencias

LangChain. (s. f.). RunnableSequence y composición LCEL. Documentación técnica oficial. Consulta: 31 de octubre de 2026.

https://reference.langchain.com/python/langchain_core/runnables/

LangChain. (s. f.). Interrupciones de LangGraph. Documentación técnica oficial. Consulta: 31 de octubre de 2026.

https://docs.langchain.com/oss/python/langgraph/interrupts

LangChain. (s. f.). Deep Agents y su arquitectura. Documentación técnica oficial. Consulta: 31 de octubre de 2026.

https://docs.langchain.com/oss/python/deepagents/overview

LangChain. (s. f.). Subagentes de Deep Agents. Documentación técnica oficial. Consulta: 31 de octubre de 2026.

https://docs.langchain.com/oss/python/deepagents/subagents

LangChain. (s. f.). Observabilidad LangSmith. Documentación técnica oficial. Consulta: 31 de octubre de 2026.

https://docs.langchain.com/langsmith/observability

Object Management Group. (s. f.). BPMN 2.0.2. Documentación técnica oficial. Consulta: 31 de octubre de 2026.

https://www.omg.org/spec/BPMN/2.0.2/

Fuentes primarias del proyecto

Veltri Tecnologic. (2026). Código fuente local, BPMN TO-BE y expediente técnico de Semana 10. Repositorio: D:/Proyectos/veltri-orquestador.

Se utilizaron ejecuciones.json, verificar.log, pytest-suite.xml, suite-summary.json, grafo-comercial.json y el registro remoto de fase 4.3. Los identificadores de archivos y líneas se relacionan en el anexo siguiente.

Las referencias describen las tecnologías; las afirmaciones sobre resultados de Veltri se sustentan en sus registros locales. Las capacidades generales del framework no se atribuyen automáticamente a esta implementación.

13 Anexos técnicos y evidencia pendiente

Tabla 12. Localización de evidencias

Archivo	Contenido utilizado
src/academic/lcel.py:10–20	Composición LCEL
src/academic/hitl.py:23–33	interrupt y checkpointer
src/academic/deep_agents.py:17–101	Adaptador, subagentes y creación
scripts/demo_academica.py	Ejecución local y exportación
src/graph/workflow.py:28–53	Grafo comercial
src/core/hitl.py:27,62	Registro y resolución de bandeja
docs/bpmn_to_be.bpmn	Proceso TO-BE original conservado
tests/test_academico.py	Ocho casos académicos
docs/evidencias/semana10-entrega-final/	Resultados y procedencia
docs/evidencias/semana10-fase43/ejecucion-unica.json	HTTP 202/403; sin traza verificada

Capturas pendientes de incorporación

C01: código y salida LCEL; C02: grafo real; C03: BPMN en modelador; C04–C05: pausa, decisión y reanudación; C06: aplicación local; C07: A1–A8 y Deep Agents; C08: pruebas; C09: traza LangSmith; C10–C13: componentes del ecosistema. Los originales no se sustituyen por los diagramas del informe.

No se aportaron capturas manuales ni se eludieron las restricciones de Computer Use. Para completar C09 se necesita una captura auténtica del proyecto y del detalle remoto con run_id, duración, spans y metadatos, sin mostrar credenciales.

Las figuras PNG del informe son diagramas documentales generados desde fuentes locales, no capturas de pantallas ni ejecuciones simuladas. Los registros originales permanecen en su carpeta de evidencia y no fueron alterados.